Universidad del Valle
Escuela de Ingeniería de Sistemas y Computación

750023C · Infraestructuras Paralelas y Distribuidas · Periodo 2026-2

Paralelismo en acción

El Juego de la Vida de Conway en versión secuencial, con hilos y con MPI

Parcial I — Informe técnico

Integrantes	**Código**
Andrés Gerardo González Rosero	

Profesor: Manuel Alejandro Pastrana Pardo, PhD

14 de octubre de 2026

Resumen

Se implementó el Juego de la Vida de Conway en Python con tres modelos de ejecución: una versión secuencial que sirve de línea base, una versión paralela con *hilos* en memoria compartida (`threading`, barreras y doble buffer) y una versión con *paso de mensajes* (MPI, `mpi4py`) que intercambia filas fantasma entre procesos vecinos. Como comparación se añadió una cuarta variante con hilos nativos de Numba (`prange`, al estilo OpenMP). Todas comparten el mismo kernel compilado con Numba, que libera el GIL, y todas producen exactamente el mismo tablero que la versión secuencial (625 de 625 corridas verificadas). Se midieron tableros de 100×100, 500×500, 1000×1000, 2000×2000, 4000×4000 con 1, 2, 4, 6, 8, 12 hilos o procesos. El mejor speedup fue 4,61× (Hilos, dinámica, 4000×4000, $P = 12$). Los resultados muestran que el paralelismo solo compensa cuando el trabajo por generación supera el costo fijo de sincronizar: en el tablero de 100×100 una generación secuencial tarda 0,007 ms, menos de lo que cuesta una sola barrera de `threading` con dos hilos (9,9 µs).

Índice

1. Introducción y objetivos **2**

2. El Juego de la Vida **2**

3. Implementación secuencial **2**

4. Estrategia de paralelización **3**
4.1. División del trabajo: bandas de filas . . . 3
4.2. Versión con hilos (memoria compartida) . . . 4
4.3. Variante de comparación: hilos nativos con `prange` . . . 5
4.4. Versión con MPI (memoria distribuida) . . . 5
4.5. Interfaz gráfica . . . 6

5. Desafíos encontrados **7**
5.1. El GIL de Python . . . 7
5.2. Condiciones de carrera y la necesidad de la barrera . . . 7
5.3. Granularidad: el costo fijo de sincronizar . . . 7
5.4. Interbloqueo en el intercambio de MPI . . . 8
5.5. Desbalance de carga . . . 8
5.6. Medir de forma confiable . . . 8

6. Metodología de medición **8**

7. Resultados **9**

8. Análisis **14**

9. Conclusiones **15**

A. Cómo ejecutar **16**

B. Tiempos completos **17**

1. Introducción y objetivos

El objetivo del parcial es aplicar los conceptos de paralelismo del curso a un problema concreto y medir si el paralelismo realmente acelera la ejecución. El Juego de la Vida es un buen caso de estudio porque es un *cómputo de plantilla* (*stencil*): cada célula se actualiza con la misma regla a partir de sus ocho vecinas, de modo que el trabajo de una generación se puede repartir por *descomposición de dominio* [1]; sin embargo, entre una generación y la siguiente hay una dependencia estricta, así que la sincronización es inevitable.

Los requisitos del enunciado y dónde se cumplen en este trabajo:

1. **Implementación secuencial** (línea base): sección 3.
2. **Implementación paralela** con hilos y con MPI: sección 4.
3. **Análisis de rendimiento** con gráficas de speedup: secciones 6 a 8.
4. **Informe**: estrategia (sección 4), desafíos (sección 5) y conclusiones (sección 9).
5. **Interfaz gráfica** que simula el juego en todas las versiones: sección 4.5.

2. El Juego de la Vida

El tablero es una matriz $N \times N$ donde cada célula está viva (1) o muerta (0). En cada generación, el estado de cada célula se recalcula a partir del número v de vecinas vivas entre sus ocho vecinas (vecindad de Moore):

- Célula viva: muere por soledad si $v < 2$, sobrevive si $v \in \{2, 3\}$ y muere por sobrepoblación si $v > 3$.
- Célula muerta: nace si $v = 3$.

Estas cuatro reglas se reducen a una sola expresión, que es la que implementa el código:

$$\text{viva}_{t+1}(i, j) \;=\; (v = 3) \;\vee\; (v = 2 \wedge \text{viva}_t(i, j)).$$

Decisiones de modelado.

- **Frontera toroidal.** El borde superior se conecta con el inferior y el izquierdo con el derecho, así ninguna célula tiene un trato especial y los planeadores cruzan los bordes. Esta decisión hace que, en MPI, los procesos formen un anillo (sección 4.4).
- **Representación.** Arreglos de NumPy de tipo `uint8` en orden por filas (*row-major*): una fila completa ocupa posiciones contiguas de memoria.
- **Doble buffer.** Se leen los estados de la generación t en un arreglo `src` y se escriben los de $t + 1$ en otro arreglo `dst`; luego se intercambian las referencias. Actualizar en el mismo arreglo es incorrecto incluso con un solo hilo (sección 5.2).
- **Patrones iniciales.** Aleatorio con 30 % de células vivas (semilla fija, para que todas las versiones partan del mismo tablero) y patrones clásicos en formato RLE: planeador, nave ligera, R-pentominó, bellota, *diehard* y el cañón de planeadores de Gosper.

3. Implementación secuencial

Todo el cálculo vive en una función *kernel* con la firma `kernel(src, dst, r0, r1)`: calcula la generación siguiente de las filas $[r_0, r_1)$ y solo escribe esas filas de `dst`. La versión secuencial la llama con $[0, N)$; las versiones paralelas la llaman con una banda de filas por hilo o proceso. Así, **las cuatro versiones ejecutan exactamente el mismo código de cálculo**, y la única

diferencia entre ellas es cómo se reparte y sincroniza el trabajo. Eso hace justa la comparación del speedup.

Listado 1: Kernel (extracto de `vida/kernels.py`).

```
@njit("void(uint8[:, ::1], uint8[:, ::1], int64, int64)", nogil=True, cache=True)
def paso_numba(src, dst, r0, r1):
    H, W = src.shape
    for i in range(r0, r1):
        arriba = i - 1 if i > 0 else H - 1          # frontera toroidal
        abajo  = i + 1 if i < H - 1 else 0
        a, b, c, d = src[arriba], src[i], src[abajo], dst[i]
        ...                                          # columnas 0 y W-1 con envoltura
        for j in range(1, W - 1):                    # columnas interiores
            n = a[j-1] + a[j] + a[j+1] + b[j-1] + b[j+1] + c[j-1] + c[j] + c[j+1]
            d[j] = 1 if (n == 3 or (n == 2 and b[j] == 1)) else 0
```

El kernel se compila con Numba [8] por dos razones. La primera es velocidad: un bucle en Python puro tarda 129,0 ms por generación en un tablero de 1000×1000, frente a 0,710 ms del kernel compilado (unas 182 veces menos). La segunda, y la más importante para este curso, es que `nogil=True` hace que el kernel **libere el GIL** mientras calcula, lo que permite que varios hilos de Python se ejecuten de verdad en paralelo (sección 5.1). La firma explícita hace que la compilación ocurra al importar el módulo y no durante la primera medición.

La corrección se verificó con pruebas automáticas (`pytest`, 45 casos): el *blinker* oscila con periodo 2, el bloque es estable, el planeador se desplaza una celda en diagonal cada 4 generaciones y da la vuelta completa al toroide, y el cañón de Gosper emite un planeador (5 células) cada 30 generaciones.

4. Estrategia de paralelización

4.1. División del trabajo: bandas de filas

Se usó **descomposición por dominio en bandas de filas contiguas**: con P trabajadores, el trabajador k procesa las filas $\lfloor kN/P \rfloor$ a $\lfloor (k+1)N/P \rfloor - 1$. La tabla 1 compara las alternativas consideradas.

Tabla 1: Alternativas de partición del tablero para P trabajadores.

	Bandas de filas (elegida)	**Bandas de columnas**	**Bloques 2D**
Vecinos por subdominio	2	2	4 (8 con esquinas)
Celdas de frontera a comunicar por generación	$2N$	$2N$	$\approx 4N/\sqrt{P}$
Datos de frontera en memoria	contiguos (una fila)	dispersos (un byte por fila)	mixtos
Uso de caché (localidad espacial)	excelente	malo	bueno
Complejidad del código	baja	baja	alta

Las columnas se descartan porque, en orden por filas, una columna está repartida en N líneas de caché distintas. Los bloques 2D comunican menos datos cuando $P > 4$ (con $P = 8$, $4N/\sqrt{8} \approx 1{,}4N$ frente a $2N$), pero obligan a hablar con más vecinos y a empaquetar columnas no contiguas. Con los $P \leq 12$ de este trabajo, el costo dominante es la *latencia* de cada sincronización y no el volumen de datos, así que las bandas de filas son la opción más simple y eficiente.

Relación cómputo/comunicación. Cada trabajador calcula N^2/P células por generación y depende de $2N$ células ajenas (las filas de frontera). La razón entre ambas es

$$\frac{\text{cómputo}}{\text{comunicación}} = \frac{N^2/P}{2N} = \frac{N}{2P},$$

que **crece con el tamaño del tablero y decrece con el número de trabajadores**. Esta relación superficie/volumen anticipa el resultado principal de la sección 7: los tableros grandes escalan mejor que los pequeños.

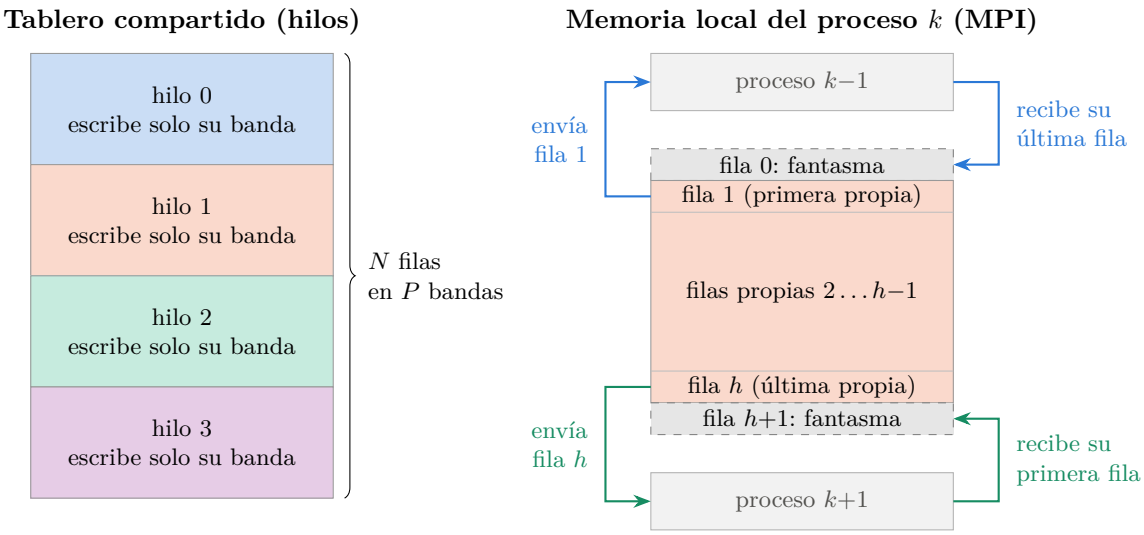


Figura 1: Izquierda: con hilos, todos leen el mismo tablero y cada uno escribe solo su banda. Derecha: con MPI, cada proceso guarda su banda más dos filas fantasma que recibe de sus vecinos en cada generación.

4.2. Versión con hilos (memoria compartida)

Esta versión corresponde al modelo SMP visto en clase [3]: varios hilos de un mismo proceso comparten la memoria y por lo tanto comparten el tablero.

Pool persistente. Los P hilos se crean una sola vez, al construir el motor, y esperan órdenes. Crear y destruir hilos en cada generación costaría más que el propio cálculo en tableros pequeños. Es la misma idea del *pool* de hilos del modelo Fork-Join [4].

Sincronización. La implementación usa tres barreras (`threading.Barrier`):

- `_inicio` y `_fin` (P+1 participantes): el hilo principal libera a los trabajadores para un lote de n generaciones y espera a que terminen.
- `_barrera_gen` (P participantes): **separa una generación de la siguiente**. Su *acción* (una función que la barrera ejecuta en un único hilo cuando todos llegaron y antes de liberarlos) intercambia los buffers. Así se garantiza que nadie empiece a leer la generación t+1 mientras otro todavía la escribe.

El tablero no necesita locks: durante una generación `src` solo se lee, y cada hilo escribe filas de `dst` que no escribe ningún otro (no hay escrituras en conflicto). La exclusión mutua se reemplaza por una partición de los datos más una barrera, que es más barato que proteger cada celda.

Listado 2: Bucle de cada hilo trabajador (extracto de `vida/motor_hilos.py`).

```
def _trabajador(self, idx):
    r0, r1 = self._bandas[idx]
    while True:
```

```
        self._inicio.wait()                    # espera la orden del hilo principal
        if self._detener:
            return
        for _ in range(self._gens_pedidas):
            src = self._buf[self._actual]
            dst = self._buf[1 - self._actual]
            if self.planificacion == "estatica":
                kernel(src, dst, r0, r1)       # suelta el GIL mientras calcula
            else:                              # planificación dinámica
                while True:
                    k = self._tomar_bloque()
                    if k >= len(self._bloques):
                        break
                    kernel(src, dst, *self._bloques[k])
            self._barrera_gen.wait()           # acción: intercambiar buffers
        self._fin.wait()

def _tomar_bloque(self):
    with self._lock_cola:                      # sección crítica mínima
        k = self._siguiente_bloque
        self._siguiente_bloque += 1
    return k
```

Planificación estática y dinámica. En la planificación *estática* cada hilo recibe una banda fija de N/P filas. En la *dinámica* el tablero se parte en $8P$ bloques y los hilos los toman de una cola compartida: un contador protegido con un `Lock`. Sin el lock, dos hilos podrían leer el mismo valor del contador (*leer–modificar–escribir* no es atómico) y procesar dos veces un bloque o saltarse otro. La planificación dinámica sirve cuando los núcleos no son iguales (por ejemplo, CPUs híbridas con núcleos de rendimiento y de eficiencia): un núcleo rápido toma más bloques que uno lento. A cambio, paga una adquisición del lock por bloque.

Manejo de errores. Si un trabajador lanza una excepción, aborta todas las barreras (`Barrier.abort()`) para que los demás hilos no queden bloqueados para siempre, y el hilo principal relanza el error.

4.3. Variante de comparación: hilos nativos con `prange`

Para separar el costo del algoritmo del costo de sincronizar desde Python, se añadió una variante que usa la *misma* partición en P bandas y el *mismo* kernel, pero delega el reparto a `numba.prange` (la capa de hilos de Numba: OpenMP, TBB o *workqueue*). Es el contraste "PThread frente a OpenMP" visto en clase [2]: el programador declara qué bucle es paralelo y el runtime hace el fork-join. Al final de cada `prange` hay una barrera implícita, pero se ejecuta en código nativo, sin pasar por el intérprete.

4.4. Versión con MPI (memoria distribuida)

En MPI cada proceso tiene su propia memoria y no ve el tablero de los demás: todo dato ajeno tiene que llegar como mensaje. La versión usa `mpi4py` [9] y se organiza así:

1. **Topología.** `Create_cart(dims=[P], periods=[True])` crea un anillo de procesos: el vecino de arriba del proceso 0 es el $P-1$. La periodicidad del anillo implementa la frontera toroidal sin casos especiales.
2. **Reparto.** El proceso 0 genera el tablero y lo reparte por filas con `Scatterv` (la variante `v` admite bandas de distinto tamaño cuando N no es múltiplo de P).

3. **Filas fantasma.** Cada proceso guarda sus h filas más dos filas fantasma (figura 1). En cada generación, antes de calcular, intercambia sus filas de frontera con los dos vecinos.
4. **Cálculo local** con el mismo kernel, sobre las filas $1 \dots h$.
5. **Recolección.** `Gatherv` reúne el tablero en el proceso 0 cuando hay que dibujarlo o verificarlo.

Listado 3: Intercambio de filas fantasma (extracto de `vida/motor_mpi.py`).

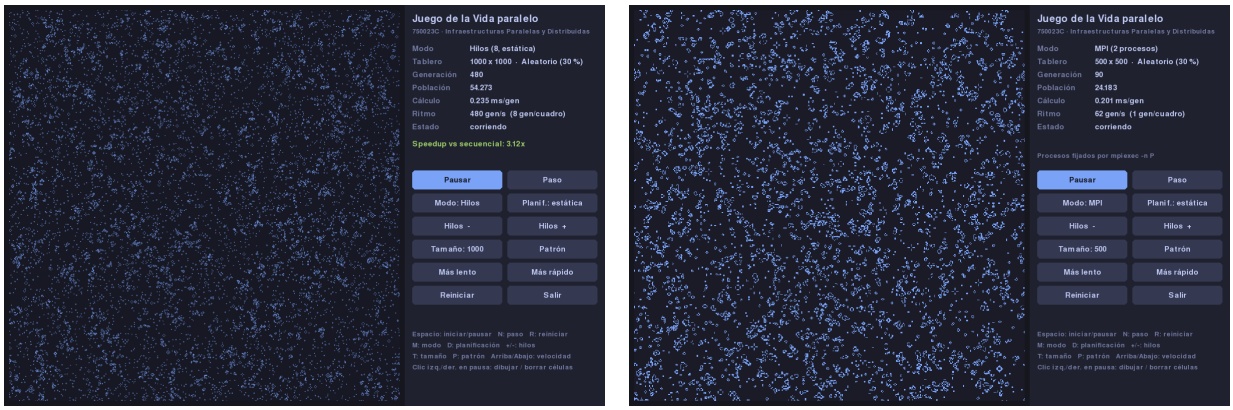
```
def _intercambiar_fronteras(self, a):
    h = self.h
    # mi primera fila sube; recibo en mi fantasma inferior la primera fila del vecino de abajo
    self.cart.Sendrecv(sendbuf=a[1], dest=self.arriba, sendtag=0,
                       recvbuf=a[h + 1], source=self.abajo, recvtag=0)
    # mi última fila baja; recibo en mi fantasma superior la última fila del vecino de arriba
    self.cart.Sendrecv(sendbuf=a[h], dest=self.abajo, sendtag=1,
                       recvbuf=a[0], source=self.arriba, recvtag=1)
```

Cada mensaje es una fila completa, que es contigua en memoria, así que no hace falta empaquetar datos. No se necesita una barrera global: el `Sendrecv` ya sincroniza a cada proceso con sus dos vecinos, que son los únicos de los que depende (sincronización punto a punto). La sección 5.4 explica por qué se usa `Sendrecv` y no un `Send` seguido de un `Recv`.

4.5. Interfaz gráfica

La interfaz está hecha con `pygame` y muestra el tablero, la generación, la población y el tiempo de cálculo por generación medido *solo* alrededor de `motor.paso()`, sin contar el dibujado. Permite cambiar en vivo entre secuencial, hilos e hilos nativos, ajustar el número de hilos y la planificación, el tamaño del tablero (100 a 2000) y el patrón, y dibujar células con el ratón. Al cambiar de modo se conserva el tablero, así que en la sustentación se puede ver el speedup en vivo: la interfaz guarda la última medición secuencial de cada tamaño y muestra la razón frente al modo actual (figura 2).

La versión MPI tiene su propia interfaz (`mpiexec -n P python -m vida.gui_mpi`): el proceso 0 abre la ventana y difunde cada orden (avanzar, reiniciar, salir) con `bcast`; después de cada lote de generaciones reúne el tablero con `Gatherv` para dibujarlo. Los demás procesos solo calculan.



(a) Hilos, 1000×1000. **(b)** MPI con 2 procesos, 500×500.

Figura 2: Interfaz gráfica. El panel muestra el tiempo de cálculo por generación y el speedup frente a la última medición secuencial del mismo tamaño.

5. Desafíos encontrados

5.1. El GIL de Python

En CPython el *Global Interpreter Lock* permite que un solo hilo ejecute código Python a la vez. Si el kernel es Python puro, los hilos se turnan y no hay aceleración. La figura 3 lo muestra con el mismo motor de hilos y tres kernels distintos sobre un tablero de 1000×1000:

- **Python puro**: speedup de 0,97 con 2 hilos y 0,94 con 8. El GIL serializa los hilos.
- **NumPy**: NumPy suelta el GIL dentro de cada operación vectorizada, pero lo retoma entre una y otra, así que hay paralelismo parcial (1,97 con 2 hilos).
- **Numba con `nogil=True`**: el kernel completo corre sin el GIL (1,81 con 2 hilos).

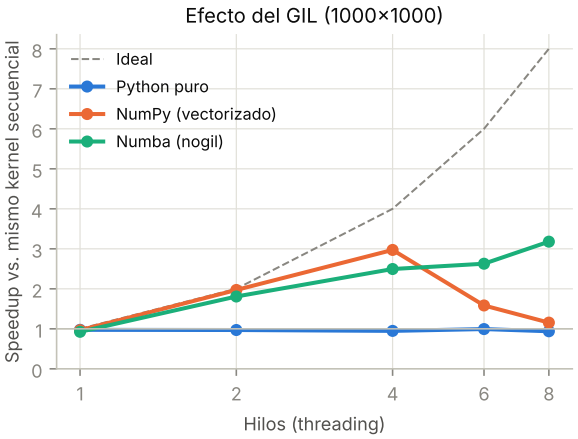


Figura 3: Speedup de la versión con hilos según el kernel.

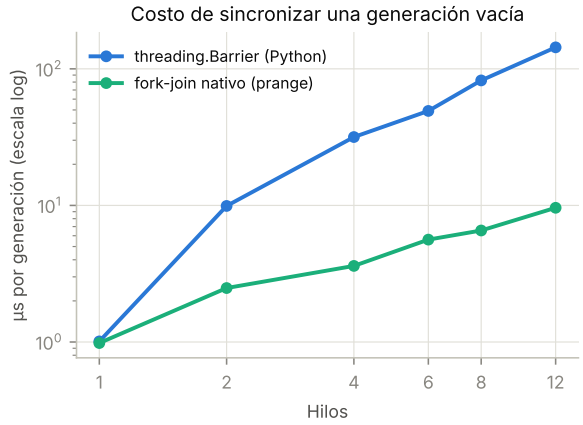


Figura 4: Costo de sincronizar una generación sin cálculo.

Por eso todas las versiones usan el kernel de Numba. Una alternativa habría sido usar procesos (`multiprocessing`) con memoria compartida; la versión MPI ya cubre el modelo de procesos. Desde Python 3.13 existe una compilación experimental *sin GIL* (*free-threaded*, PEP 703) en la que incluso el kernel de Python puro escalaría con hilos.

5.2. Condiciones de carrera y la necesidad de la barrera

Para comprobar que la sincronización es necesaria, el script `bench/demo_condicion_carrera.py` ejecuta 200 generaciones de un tablero de 300×300 con 4 hilos en tres variantes y cuenta las células que difieren del resultado secuencial:

- **Sin barrera entre generaciones**: un hilo que termina rápido empieza la generación siguiente leyendo las filas de frontera de su vecino, que todavía no las ha actualizado. Resultado: entre 11.875 y 12.913 células erróneas, *distintas en cada uno de los 5 intentos*. Ese no determinismo es la firma de una condición de carrera.
- **Actualización en el mismo arreglo** (sin doble buffer), incluso con un solo hilo: las células ya actualizadas contaminan el conteo de sus vecinas. Resultado: 37.463 células erróneas.
- **Versión correcta** (barrera + doble buffer): 0 células distintas en todos los intentos.

5.3. Granularidad: el costo fijo de sincronizar

Cada generación paga una barrera, sin importar el tamaño del tablero. La figura 4 mide ese costo con un kernel vacío: con `threading.Barrier` cuesta 9,9 µs con 2 hilos y 143,6 µs con 12,

porque cada espera implica adquirir un lock, una variable de condición y el GIL para despertar a cada hilo. El fork-join nativo de `prange` cuesta 2,5 y 9,6 µs respectivamente. Comparado con los 0,007 ms que tarda una generación secuencial de 100×100, *sincronizar cuesta más que calcular*. Este es el desafío que más influyó en los resultados.

La planificación dinámica agrava el problema en tableros pequeños: los $8P$ bloques implican $8P$ adquisiciones del lock por generación, cada una con su llamada desde Python.

5.4. Interbloqueo en el intercambio de MPI

La forma ingenua del intercambio es que cada proceso haga `Send` a su vecino y luego `Recv`. Si los envíos son bloqueantes y todos los procesos envían a la vez, todos esperan a que alguien reciba y nadie llega al `Recv`: se cumplen las condiciones de Coffman [5] con una espera circular alrededor del anillo. Que funcione o no depende de si la implementación de MPI almacena el mensaje en un buffer interno (los mensajes pequeños sí, los grandes no), lo que hace el error intermitente. `Sendrecv` resuelve el problema porque la biblioteca empareja internamente el envío y la recepción.

5.5. Desbalance de carga

En la planificación estática el tiempo de cada generación es el del hilo *más lento*: los demás esperan en la barrera. El motor registra el tiempo de cómputo de cada hilo; la figura 10 muestra la razón entre el hilo más lento y el promedio. En CPUs híbridas (núcleos P y E, como las Intel de 12.ª generación en adelante) o con el sistema operativo moviendo hilos entre núcleos, esta razón crece y la planificación dinámica compensa.

5.6. Medir de forma confiable

- La compilación JIT se hace al importar (firma explícita), nunca dentro del tiempo medido.
- Se ejecutan 3 generaciones de calentamiento antes de cronometrar.
- La creación del pool de hilos y el reparto/recolección de MPI (`Scatterv`/`Gatherv`) quedan fuera del tiempo; la sincronización y el intercambio de fronteras de *cada* generación sí se miden, porque son parte del algoritmo.
- El número de generaciones se calibra por tamaño (corridas secuenciales de ≈0,6 s) y es el mismo para todas las versiones de ese tamaño.
- Cada configuración se repite 5 veces y se reporta la mediana, menos sensible a interferencias que el promedio.
- Cada corrida paralela se compara celda por celda con el resultado secuencial.

6. Metodología de medición

Tabla 2: Entorno de medición (generado automáticamente).

Procesador	Intel Core i5-12450H (4 núcleos P con HT + 4 núcleos E)
Memoria	8 GB DDR5-4800 en un solo canal
Núcleos físicos / lógicos	8 / 12
Sistema operativo	Linux 7.2.9-arch1-1
Python / NumPy / Numba	3.12.15 / 2.5.4 / 0.68.0
Biblioteca MPI	MPICH Version: 5.0.2
Tableros	100×100, 500×500, 1000×1000, 2000×2000, 4000×4000
Hilos / procesos (P)	1, 2, 4, 6, 8, 12
Repeticiones por configuración	5 (se reporta la mediana)

Métricas: tiempo por generación T_P (ms), **speedup** $S_P = T_{\text{sec}}/T_P$ y **eficiencia** $E_P = S_P/P$. La línea base T_{sec} es siempre la versión secuencial con el mismo kernel, no la versión paralela con $P = 1$. Por eso el speedup con $P = 1$ mide directamente el sobrecosto de cada mecanismo de paralelización. Todo el proceso es reproducible con `python bench/ejecutar_todo.py`.

7. Resultados

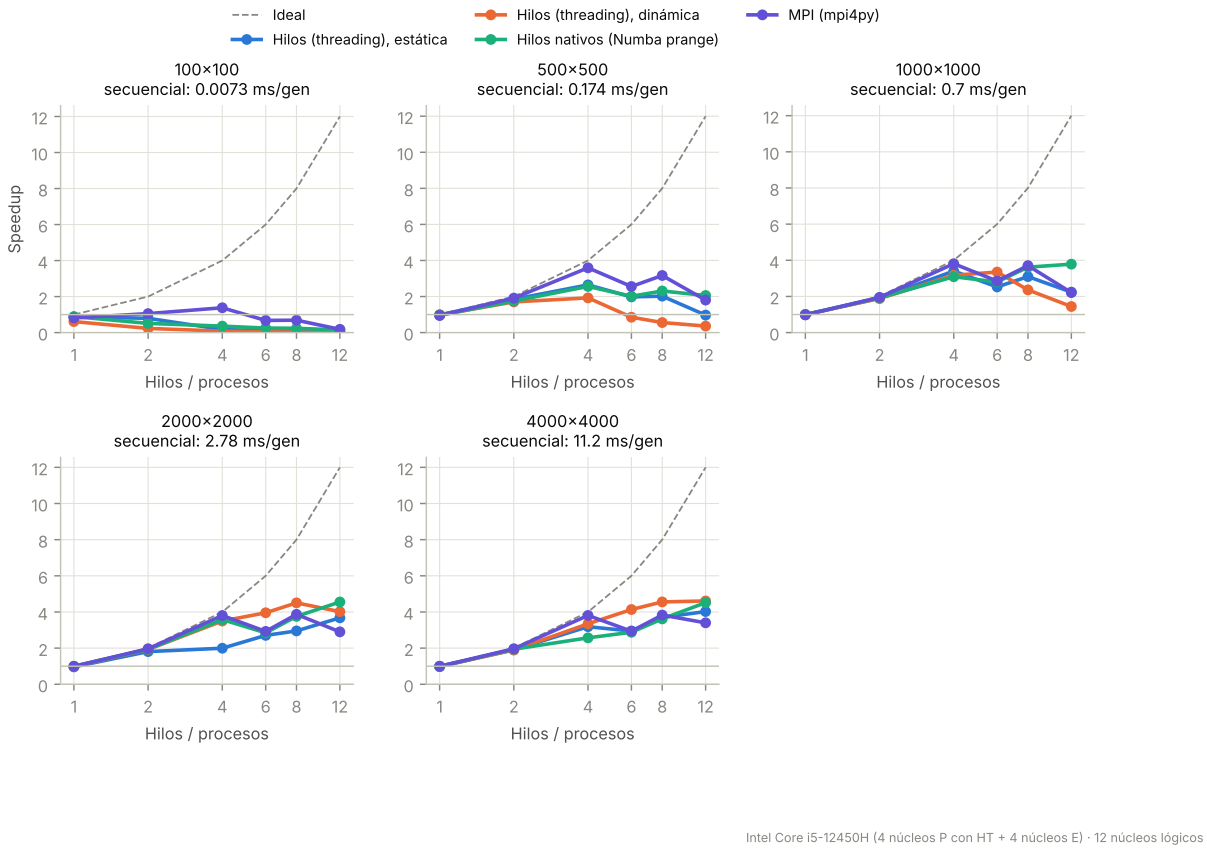


Figura 5: Speedup de cada implementación por tamaño de tablero. La línea punteada es el speedup ideal ($S_P = P$).

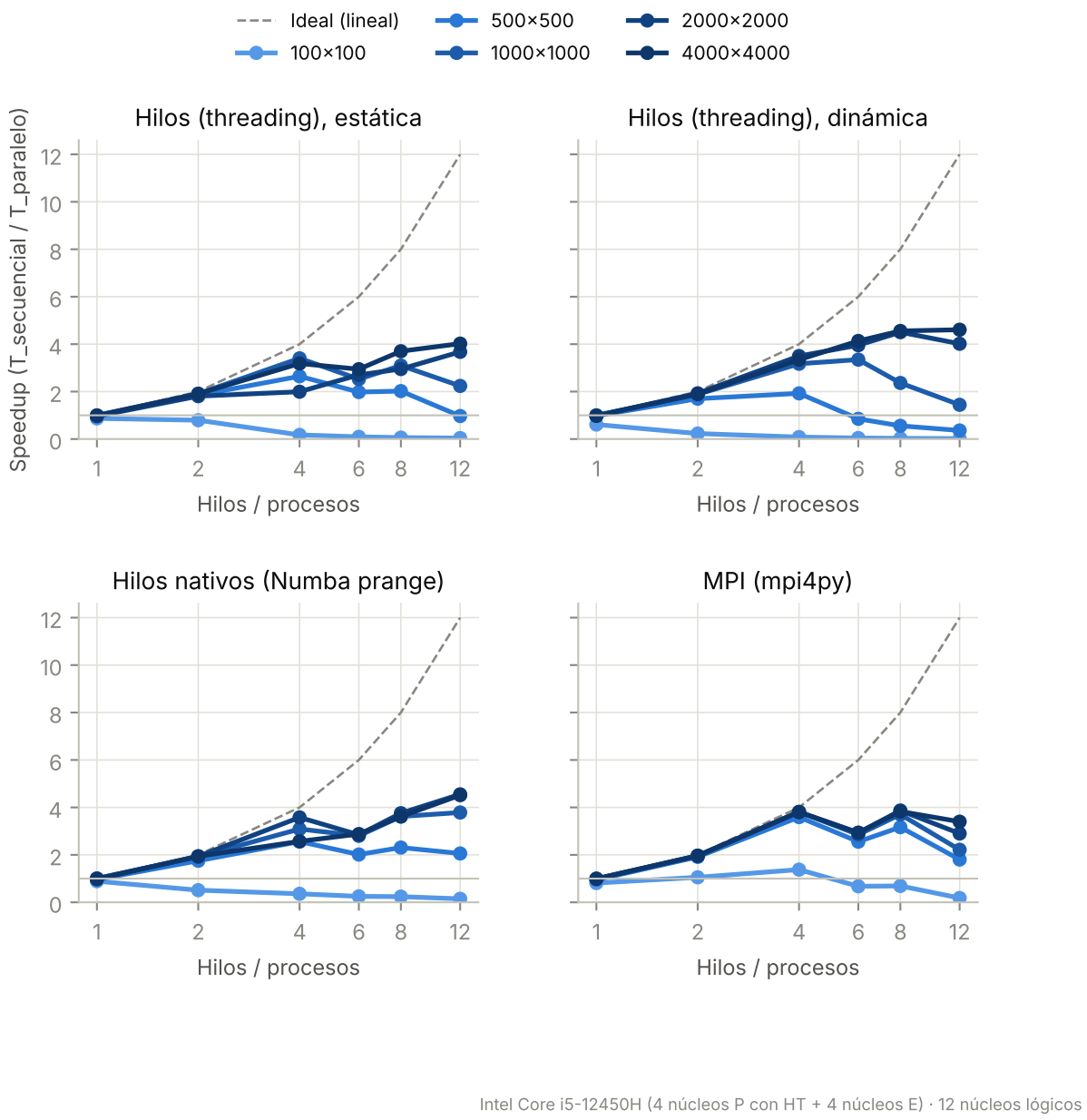


Figura 6: Speedup frente a número de hilos o procesos, con una línea por tamaño de tablero.

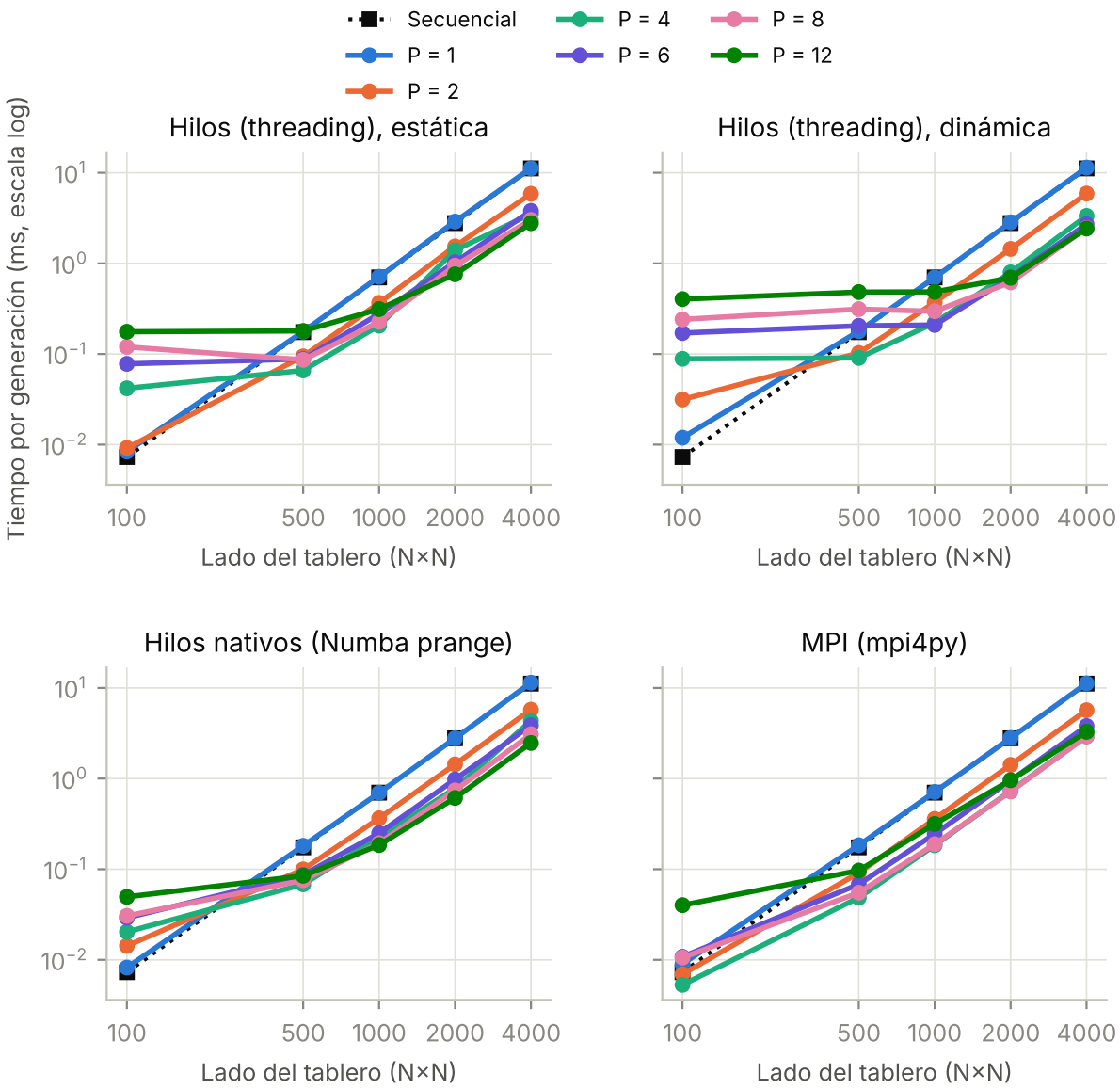


Figura 7: Tiempo por generación frente al tamaño del tablero (escala logarítmica en ambos ejes). Con trabajo $O(N^2)$, las curvas deberían ser rectas de pendiente 2; las desviaciones en tableros pequeños son el costo fijo de sincronizar.

Tabla 3: Speedup (mediana de 5 repeticiones). En negrita, el mejor valor de cada fila cuando supera 1.

Tablero	Implementación	$P = 1$	$P = 2$	$P = 4$	$P = 6$	$P = 8$	$P = 12$
100×100 sec.: 0,007 ms	Hilos, estática	0,87	0,79	0,17	0,09	0,06	0,04
	Hilos, dinámica	0,61	0,23	0,08	0,04	0,03	0,02
	Hilos nativos (prange)	0,89	0,51	0,36	0,25	0,24	0,15
	MPI	0,81	1,05	**1,38**	0,68	0,69	0,18
500×500 sec.: 0,174 ms	Hilos, estática	0,99	1,85	**2,65**	1,98	2,02	0,97
	Hilos, dinámica	0,98	1,70	**1,93**	0,85	0,56	0,36
	Hilos nativos (prange)	0,96	1,75	**2,56**	2,01	2,31	2,06
	MPI	0,95	1,91	**3,59**	2,56	3,17	1,80
1000×1000 sec.: 0,700 ms	Hilos, estática	0,98	1,91	**3,40**	2,53	3,10	2,24
	Hilos, dinámica	1,00	1,88	3,17	**3,35**	2,36	1,44
	Hilos nativos (prange)	1,00	1,92	3,09	2,80	3,62	**3,79**
	MPI	0,98	1,95	**3,81**	2,85	3,70	2,21
2000×2000 sec.: 2,784 ms	Hilos, estática	0,96	1,81	1,99	2,71	2,95	**3,68**
	Hilos, dinámica	0,98	1,93	3,50	3,95	**4,50**	4,02
	Hilos nativos (prange)	1,00	1,94	3,58	2,84	3,76	**4,55**
	MPI	0,99	1,97	3,80	2,93	**3,87**	2,90
4000×4000 sec.: 11,169 ms	Hilos, estática	1,00	1,91	3,19	2,94	3,70	**4,02**
	Hilos, dinámica	0,98	1,90	3,34	4,14	4,56	**4,61**
	Hilos nativos (prange)	0,98	1,94	2,57	2,88	3,62	**4,52**
	MPI	0,99	1,97	3,81	2,94	**3,83**	3,40

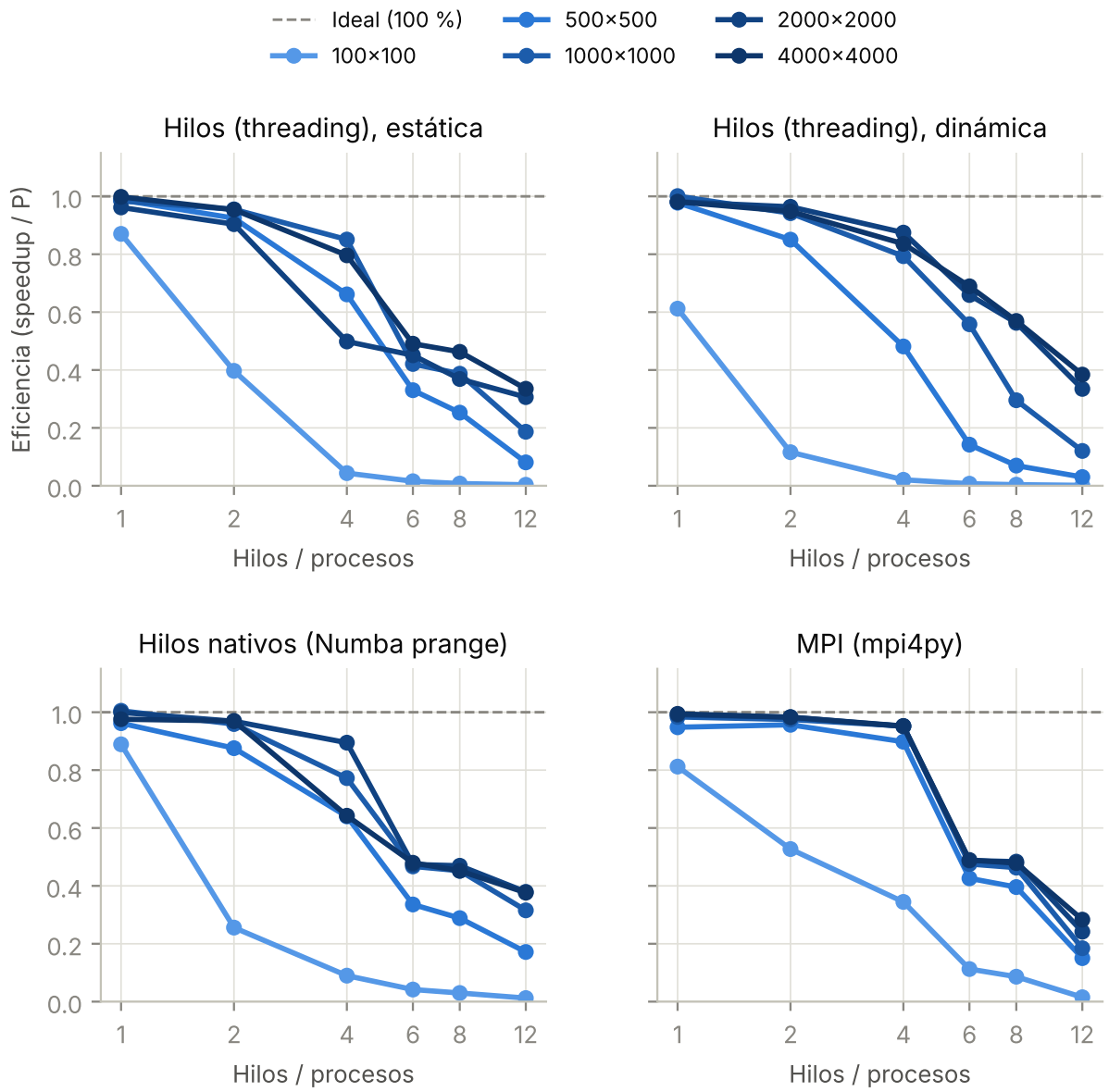


Figura 8: Eficiencia $E_P = S_P/P$.

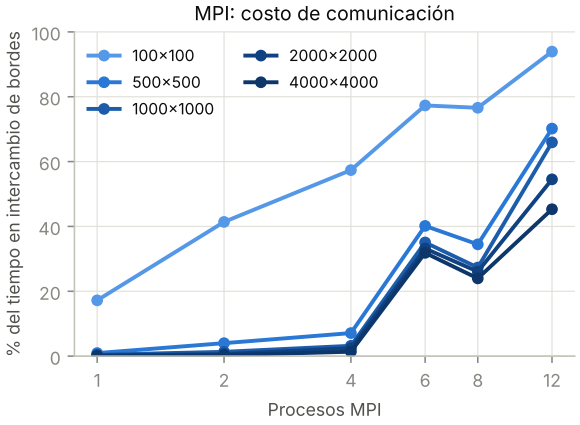


Figura 9: MPI: porcentaje del tiempo dentro de `Sendrecv` (transferencia más espera al vecino).

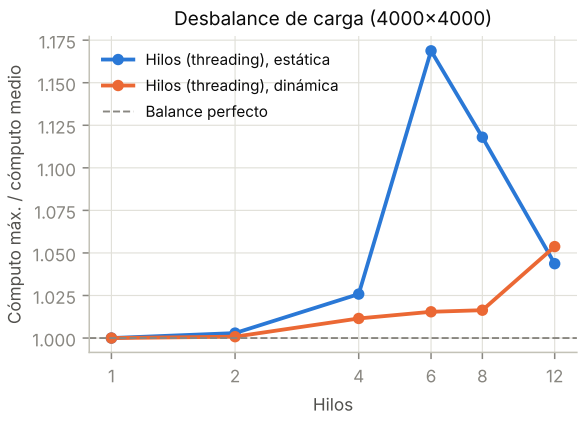


Figura 10: Desbalance de carga entre hilos en el tablero más grande.

8. Análisis

1. El tamaño del tablero decide si el paralelismo vale la pena. En 100×100 una generación secuencial tarda 0,007 ms, del mismo orden que una sola barrera, así que los hilos son *más lentos* que la versión secuencial (mejor speedup: 0,87 con `threading` y 0,89 con hilos nativos). La única ganancia en ese tamaño fue la de MPI (1,38× con $P = 4$). Sus procesos no comparten un GIL, y el `Sendrecv` entre procesos de la misma máquina se resuelve en código nativo sobre memoria compartida, sin pasar por el intérprete. Desde 500×500 todas las versiones aceleran (mejor: 3,59×, MPI, $P = 4$), porque el trabajo por generación crece como N^2 mientras que el costo de sincronizar se mantiene casi constante. Es lo que anticipa la razón $N/2P$ de la sección 4.

2. La CPU híbrida explica la caída en $P = 6$. Todas las curvas de speedup suben casi linealmente hasta $P = 4$ y *caen* en $P = 6$. Por ejemplo, en 1000×1000 MPI pasa de 3,81 a 2,85, y los hilos con planificación estática de 3,40 a 2,53. El procesador (Intel Core i5-12450H (4 núcleos P con HT + 4 núcleos E)) tiene 4 núcleos de rendimiento (P) y 4 de eficiencia (E), más lentos. Con $P \leq 4$ el sistema operativo ubica a cada trabajador en un núcleo P. Con $P = 6$, algunos trabajadores caen en núcleos E o comparten un núcleo P mediante *hyper-threading*. Con la partición estática todos reciben la misma cantidad de filas, y en cada generación todos esperan al más lento. Las mediciones lo confirman:

- En MPI, el tiempo de *cómputo* del proceso más lento respecto al promedio sube de 1,01 con $P = 4$ a 1,49 con $P = 6$.
- La fracción del tiempo que los procesos pasan dentro de `Sendrecv` (esperando al vecino lento) salta de 3 % a 35 %.

La **planificación dinámica** corrige justamente esto: los núcleos rápidos toman más bloques. En $P = 6$ logra 3,95 frente a 2,71 de la estática en 2000×2000, y 4,14 frente a 2,94 en 4000×4000. El mejor resultado de todo el trabajo (4,61×) es de la planificación dinámica con 12 hilos. En tableros pequeños pasa lo contrario: los $8P$ bloques implican más adquisiciones del lock que trabajo útil, y la dinámica es la peor opción.

3. Un modelo simple explica el comportamiento de `threading`. Si se suma al tiempo ideal el costo de sincronización medido aparte, $T_P \approx T_{\text{sec}}/P + s(P)$, para 1000×1000 con $P = 4$ el speedup predicho es 3,39 ($T_{\text{sec}}/4 = 0,175$ ms más $s(4) = 31,7$ µs), frente a 3,40 medido. Con núcleos homogéneos, toda la diferencia con el ideal es el costo de la barrera. Con $P = 12$ el mismo modelo predice 3,46, pero se midió 2,24. La diferencia restante es la heterogeneidad de núcleos del punto anterior, que el modelo no incluye.

4. Hilos nativos frente a `threading`. Las dos versiones usan la misma partición y el mismo kernel; solo cambia quién sincroniza. En este equipo una barrera de `threading` cuesta 9,9 µs con 2 hilos, pero 143,6 µs con 12, unas 15 veces más que el fork-join nativo (9,6 µs). Con pocos hilos la diferencia casi no se nota (en 500×500 con $P = 4$: 2,65 frente a 2,56). Con muchos hilos sí: en 1000×1000 con $P = 12$, 3,79 con hilos nativos frente a 2,24 con `threading`. El algoritmo escala, y lo que frena a `threading` con muchos hilos es sincronizar desde el intérprete.

5. Ley de Amdahl y fracción serial experimental. La ley de Amdahl, $S_P = 1/((1-f)+f/P)$, acota el speedup según la fracción paralelizable f [2]. A partir del speedup medido se puede estimar la fracción serial *efectiva* con la métrica de Karp–Flatt [7],

$$e = \frac{1/S_P - 1/P}{1 - 1/P}.$$

En la tabla 4, para tableros de 1000×1000 o más y $P \leq 4$, e queda por debajo de 0,1 en casi todas las configuraciones (hay dos excepciones aisladas, probablemente por ruido del sistema): el

algoritmo es casi perfectamente paralelizable. En $P = 6$, e salta de golpe. Una parte serial real del programa no puede aparecer de repente al agregar dos trabajadores, así que ese salto no es serialidad sino el sobrecosto de sincronizar con núcleos de distinta velocidad (punto 2). En los tableros grandes, la planificación dinámica mantiene el e más bajo para $P \geq 6$.

Tabla 4: Fracción serial efectiva de Karp–Flatt (valores bajos son mejores).

Tablero	Implementación	$P = 2$	$P = 4$	$P = 6$	$P = 8$	$P = 12$
500×500	Hilos, estática	0,082	0,171	0,406	0,423	1,034
	Hilos, dinámica	0,176	0,359	1,208	1,906	2,928
	Hilos nativos (prange)	0,142	0,187	0,396	0,352	0,439
	MPI	0,045	0,038	0,269	0,218	0,514
1000×1000	Hilos, estática	0,047	0,058	0,275	0,226	0,396
	Hilos, dinámica	0,062	0,087	0,158	0,341	0,665
	Hilos nativos (prange)	0,043	0,098	0,228	0,173	0,197
	MPI	0,026	0,017	0,221	0,166	0,402
2000×2000	Hilos, estática	0,106	0,335	0,244	0,244	0,206
	Hilos, dinámica	0,037	0,048	0,103	0,111	0,181
	Hilos nativos (prange)	0,032	0,039	0,222	0,161	0,149
	MPI	0,017	0,017	0,210	0,153	0,285
4000×4000	Hilos, estática	0,048	0,085	0,207	0,166	0,180
	Hilos, dinámica	0,055	0,065	0,090	0,108	0,146
	Hilos nativos (prange)	0,031	0,186	0,217	0,173	0,151
	MPI	0,017	0,017	0,209	0,156	0,230

6. MPI: comunicación frente a cómputo. Con núcleos homogéneos ($P \leq 4$) la comunicación es casi gratis: en 4000×4000 con $P = 4$ los procesos pasan el 1 % del tiempo en `Sendrecv`. Con $P = 12$ ese porcentaje sube a 45 % (94 % en 100×100; figura 9). Ese tiempo incluye la espera al vecino más lento, así que en este equipo mide sobre todo desbalance y no volumen de datos. MPI fue la mejor versión con $P = 4$ en 500×500 y 1000×1000 porque no paga el GIL ni una barrera global: cada proceso solo espera a sus dos vecinos. Además, es la única versión que podría escalar a varias máquinas sin cambiar el algoritmo.

7. Memoria y caché. Una generación lee un tablero y escribe otro: $2N^2$ bytes. Con $N = 4000$ son 32 MB por generación, bastante más que los 12 MB de caché L3 del procesador, así que el kernel queda limitado por el ancho de banda de la RAM (8 GB DDR5-4800 en un solo canal). Aunque 4000×4000 tiene cuatro veces más trabajo que 2000×2000, su mejor speedup (4,61) es prácticamente el mismo (4,55): agregar hilos no aumenta el ancho de banda de memoria. Es el "muro de memoria" de las arquitecturas multinúcleo visto en clase [2].

9. Conclusiones

1. **El paralelismo funciona cuando el trabajo por generación es suficiente.** El mejor resultado fue 4,61× (Hilos, dinámica, 4000×4000, $P = 12$). En 100×100 los hilos son más lentos que la versión secuencial, y solo MPI logró una ganancia modesta (1,38×).
2. **Con núcleos homogéneos el algoritmo escala casi linealmente.** Hasta $P = 4$ el speedup está entre 3 y 3,8 (de un máximo de 4) en casi todas las configuraciones con tableros de 1000×1000 o más, y la diferencia con el ideal se explica casi exactamente por el costo medido de la barrera.
3. **En una CPU híbrida, el reparto estático se paga.** Pasar de 4 a 6 trabajadores *reduce* el speedup con planificación estática y en MPI, porque todos esperan al núcleo más lento.

La planificación dinámica (cola de bloques con un lock) recupera ese rendimiento y dio el mejor resultado del trabajo.

4. **En Python, los hilos solo sirven si el cálculo suelta el GIL.** Con un kernel en Python puro, los hilos no aceleran nada. Con Numba (`nogil=True`) sí hay paralelismo real.
5. **La corrección depende de dos decisiones**: el doble buffer y la barrera entre generaciones. Sin cualquiera de las dos el resultado es incorrecto, y sin la barrera además es no determinista.
6. **Hilos frente a MPI.** Los hilos comparten el tablero y no copian datos, pero en Python pagan una sincronización cara que crece con el número de hilos. MPI paga dos mensajes por generación y por proceso, pero solo sincroniza con sus vecinos y escala a memoria distribuida.
7. **Trabajo futuro**: planificación dinámica o reparto proporcional a la velocidad de cada núcleo también en MPI; intercambio no bloqueante (`Isend`/`Irecv`) para solapar la comunicación con el cálculo de las filas interiores; filas fantasma más anchas para sincronizar cada k generaciones; y probar Python sin GIL (3.13t/3.14t).

A. Cómo ejecutar

```
# Entorno (conda/mamba): incluye MPICH y mpi4py
mamba env create -f environment.yml && mamba activate vida-paralela

python -m vida.gui                                  # GUI: secuencial / hilos / nativo
python -m vida.gui --modo hilos --hilos 8 --tamano 1000 --iniciar
mpiexec -n 4 python -m vida.gui_mpi --tamano 1000  # GUI de la versión MPI

python -m pytest -q                                  # pruebas de correctitud
python bench/demo_condicion_carrera.py               # demostración sin barrera
python bench/ejecutar_todo.py                        # todas las mediciones y gráficas
python informe/generar_datos.py && cd informe && latexmk -pdf informe.tex
```

B. Tiempos completos

Tabla 5: Tiempo por generación en milisegundos (mediana).

Tablero	Gens.	Implementación	$P=1$	$P=2$	$P=4$	$P=6$	$P=8$	$P=12$
100×100	20000	Secuencial	0,0073					
		Hilos, estática	0,0084	0,0092	0,0418	0,0779	0,1198	0,1760
		Hilos, dinámica	0,0119	0,0315	0,0885	0,1704	0,2409	0,4033
		Hilos nativos (prange)	0,0082	0,0143	0,0203	0,0291	0,0306	0,0495
		MPI	0,0090	0,0069	0,0053	0,0108	0,0106	0,0401
500×500	3514	Secuencial	0,1743					
		Hilos, estática	0,1765	0,0943	0,0659	0,0880	0,0862	0,1797
		Hilos, dinámica	0,1782	0,1025	0,0905	0,2045	0,3126	0,4823
		Hilos nativos (prange)	0,1812	0,0995	0,0681	0,0866	0,0755	0,0847
		MPI	0,1839	0,0911	0,0485	0,0682	0,0550	0,0967
1000×1000	895	Secuencial	0,6998					
		Hilos, estática	0,7111	0,3664	0,2056	0,2771	0,2259	0,3125
		Hilos, dinámica	0,6991	0,3715	0,2205	0,2090	0,2961	0,4848
		Hilos nativos (prange)	0,6963	0,3648	0,2265	0,2496	0,1934	0,1849
		MPI	0,7114	0,3589	0,1837	0,2453	0,1889	0,3160
2000×2000	213	Secuencial	2,7843					
		Hilos, estática	2,8949	1,5402	1,3961	1,0292	0,9434	0,7575
		Hilos, dinámica	2,8373	1,4443	0,7958	0,7040	0,6182	0,6932
		Hilos nativos (prange)	2,7863	1,4370	0,7779	0,9791	0,7412	0,6117
		MPI	2,8085	1,4163	0,7319	0,9518	0,7196	0,9601
4000×4000	53	Secuencial	11,1692					
		Hilos, estática	11,1885	5,8541	3,5067	3,7926	3,0173	2,7768
		Hilos, dinámica	11,3836	5,8917	3,3398	2,7011	2,4514	2,4235
		Hilos nativos (prange)	11,4455	5,7581	4,3469	3,8800	3,0883	2,4731
		MPI	11,2330	5,6817	2,9322	3,8040	2,9170	3,2853

Referencias

[1] M. A. Pastrana, *Técnicas de descomposición en algoritmos paralelos*, 750023C, Universidad del Valle, 2026.

[2] M. A. Pastrana, *Introducción a la programación en paralelo y su importancia en computación*, 750023C, Universidad del Valle, 2026.

[3] M. A. Pastrana, *Apuntes sobre arquitecturas SMP y SIMD*, 750023C, Universidad del Valle, 2026.

[4] S. Calvo *et al.*, *Algoritmo paralelo Fork-Join*, exposición, 750023C, Universidad del Valle, 2026.

[5] R. H. Arpaci-Dusseau y A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, caps. 26–32, 2018.

[6] M. Gardner, "The fantastic combinations of John Conway's new solitaire game 'life"', *Scientific American*, 223(4), 1970.

[7] A. H. Karp y H. P. Flatt, "Measuring parallel processor performance", *Communications of the ACM*, 33(5), 1990.

[8] S. K. Lam, A. Pitrou y S. Seibert, "Numba: a LLVM-based Python JIT compiler", *Proc. LLVM-HPC*, 2015.

[9] L. Dalcin y Y.-L. L. Fang, "mpi4py: Status update after 12 years of development", *Computing in Science & Engineering*, 23(4), 2021.